

Nash Equilibrium on Rent Division Game

A Game Theory Approach

Orazalinova Malika

Abenova Madina

Uralina Agila

Game Theory Project

April 28, 2026

Presentation Outline

-
1. What is Nash Equilibrium?
 2. IDSDS / IDWDS Concepts
 3. The Rent Division Problem
 4. Finding the Nash Equilibrium
 5. Python Implementation
 6. Results and Conclusion
 7. Team Demonstration
-

Simple Definition

Nash Equilibrium

A situation where **no one wants to change** their choice because everyone is already doing the best they can.

Example

You and a friend choose a restaurant. If neither wants to switch, that's a Nash Equilibrium.

The Mathematical Formula

$$\pi_i(s_i^*, s_{-i}^*) \geq \pi_i(s_i, s_{-i}^*) \quad \forall s_i \in S_i$$

Meaning:

- Your current payoff
- Any other option
- No improvement possible

Key Insight

Every player is playing their **best response** to others.

IDSDS / IDWDS: Basic Definitions

Notation

- $IDSDS(G) \subseteq S(G)$: strategies surviving iterated deletion of **strictly** dominated strategies.
- $IDWDS(G) \subseteq S(G)$: strategies surviving iterated deletion of **weakly** dominated strategies.
- $NE(G)$: set of Nash equilibria of G .

Main Question

How are these sets related to each other and to Nash Equilibrium?

IDSDS / IDWDS: Inclusion Relations

Theorem (Inclusion Relations)

$$IDSDS(G) \subseteq NE(G) \quad \text{and} \quad IDWDS(G) \subseteq NE(G)$$

Key Observation

- Every strategy profile from **IDSDS** is always a Nash equilibrium:

$$NE(G) \supseteq IDSDS(G)$$

- However, **some** Nash equilibria may **not** survive **IDWDS**.

⇒ IDSDS is “safer” than IDWDS for equilibrium preservation.

Special Cases: Unique Survivors

When deletion leaves exactly one strategy profile

Let G be an ordinal strategic-form game and $s \in S(G)$ a strategy profile in G .

(A) Unique IDSDS Outcome

If $\text{IDSDS}(G) = \{s\}$, then s is a **strict** Nash equilibrium.

$$\text{IDSDS}(G) = \{s\} \Rightarrow s \text{ is strict NE}$$

(B) Unique IDWDS Outcome

If $\text{IDWDS}(G) = \{s\}$, then s is a Nash equilibrium (not necessarily strict).

$$\text{IDWDS}(G) = \{s\} \Rightarrow s \text{ is NE}$$

Backward Induction in Extensive-Form Games

Definition (Backward Induction (BI))

BI is a recursive procedure solving finite **perfect information** extensive-form games:

- 1 Start at the **last decision nodes** (subgames of length 1).
- 2 At each such node, choose the action maximizing the mover's payoff.
- 3 Replace the subgame with the resulting payoff vector.
- 4 Repeat backward until the root is reached.

Outcome

Let $b(G)$ denote the strategy profile generated by backward induction.

Backward Induction and Nash Equilibrium

Theorem ($BI \subseteq NE$)

For any finite perfect-information game G :

$$b(G) \subseteq NE(G)$$

The backward induction outcome is always a Nash equilibrium.

Stronger Result: Subgame Perfect Equilibrium (SPE)

Backward induction produces a **subgame perfect** Nash equilibrium:

$$b(G) = SPE(G) \quad (\text{for perfect-information games without chance})$$

$$SPE(G) \subseteq NE(G)$$

Key Formula

Every BI profile is an NE, but not every NE is obtained by BI (unless

Rent Division Game: Settings

Players & Contract

Three students sign a contract with landlord **Mustafa**:

- Malika
- Agila
- Madina

Total rent: 300,000 tenge

Room Characteristics

- 1 **Room 1:** Large, window, balcony
- 2 **Room 2:** Medium, perfect Wi-Fi
- 3 **Room 3:** Small, individual kitchen

Problem

How to divide the rent so that **no one wants to switch rooms?**

Private Valuations & Preferences

Student	Room 1	Room 2	Room 3
Malika	150,000	100,000	50,000
Agila	120,000	140,000	40,000
Madina	130,000	60,000	110,000

Table: Private valuations (how much each room is worth to each student)

Malika:

$1 \succ 2 \succ 3$

Agila:

$2 \succ 1 \succ 3$

Madina:

$1 \succ 3 \succ 2$

Conflict

Both Malika and Madina want **Room 1** as their first choice!

Rent Allocation & Utility Formula

Rent per Room

Room	Rent
Room 1	120,000
Room 2	100,000
Room 3	80,000

Budget Check

$$120 + 100 + 80 = 300 \text{ (thousand tenge)}$$

✓ **Exactly 300,000**

$$\text{Utility} = \text{Valuation} - \text{Rent}$$

Student	Room	Valuation	Rent	Utility
Malika	Room 1	150,000	120,000	+30,000
Agila	Room 2	140,000	100,000	+40,000
Madina	Room 3	110,000	80,000	+30,000

Strategy Tables: Madina's Choices

Madina chooses Room 1 (Utility: 10)

Malika \ Agila	Room 1	Room 2	Room 3
Room 1	(30,0,10)	(30,40,10)	(30,-40,10)
Room 2	(0,0,10)	(0,40,10)	(0,-40,10)
Room 3	(-30,0,10)	(-30,40,10)	(-30,-40,10)

Madina chooses Room 2 (Utility: -40)

Malika \ Agila	Room 1	Room 2	Room 3
Room 1	(30,0,-40)	(30,40,-40)	(30,-40,-40)
Room 2	(0,0,-40)	(0,40,-40)	(0,-40,-40)
Room 3	(-30,0,-40)	(-30,40,-40)	(-30,-40,-40)

Nash Equilibrium: Identification & Proof

Madina chooses Room 3 (Utility: 30)

Malika \ Agila	Room 1	Room 2	Room 3
Room 1	(30,0,30)	(30,40,30)	(30,-40,30)
Room 2	(0,0,30)	(0,40,30)	(0,-40,30)
Room 3	(-30,0,30)	(-30,40,30)	(-30,-40,30)

Nash Equilibrium Found!

(Malika: Room 1, Agila: Room 2, Madina: Room 3)

Payoff Vector: (30, 40, 30)

Stability Proof

- **Malika:** Gets 30. Alternatives: 0 or less → No switch
- **Agila:** Gets 40. Alternatives: 0 or less → No switch
- **Madina:** Gets 30. Alternatives: 10 or less → No switch

Python Code Structure

Input Data:

- 3 players
- 3 rooms
- Valuations matrix
- Rent prices

Core Functions:

- `get_utility()`
- `is_nash()`
- `run_analysis()`

Algorithm

1. Calculate all utilities
2. Check each strategy profile
3. Test unilateral deviations
4. Output Nash Equilibrium

is_nash() Function Logic

```
def is_nash(m_room, a_room, md_room):  
    # Get current utilities  
    u_m = get_utility("Malika", m_room)  
    u_a = get_utility("Agila", a_room)  
    u_md = get_utility("Madina", md_room)  
  
    # Check if anyone can improve  
    if any(get_utility("Malika", r) > u_m): return False  
    if any(get_utility("Agila", r) > u_a): return False  
    if any(get_utility("Madina", r) > u_md): return False  
  
    return True
```

Program Output

FINAL RESULT: NASH EQUILIBRIUM IDENTIFIED

Optimal Strategy:

Malika → Room 1 (Utility: 30)

Agila → Room 2 (Utility: 40)

Madina → Room 3 (Utility: 30)

Payoff Vector: (30, 40, 30)

✓ Stable allocation found!

Final Allocation Summary

Student	Room	Utility	Happy?
Malika	Room 1	30	Yes
Agila	Room 2	40	Yes
Madina	Room 3	30	Yes

"The best result comes from everyone in the group doing what is best for himself and the group."

— John Nash

Key Takeaways

- Nash Equilibrium provides **stability** in strategic interactions
- IDSDS always preserves Nash equilibria; IDWDS may eliminate some
- Successfully applied to **real-world rent division**
- Python **automates** the equilibrium detection
- All three students benefit from the stable allocation

Project Success

A practical demonstration of game theory solving everyday problems!

Thank You!

Game theory: Not just abstract math, but a tool for everyday life!